R Module 4: Tables and Formatted Output

BMSC 620 | R Workflow Series

Emile Latour

What We're Learning Today

By the end of this module, you will be able to:

1. Use `knitr::kable()` to produce clean simple tables
2. Use the `gt` package to build **publication-quality** formatted tables
3. Apply column labels, number formatting, and captions inside tables
4. Use `gtsummary::tbl_summary()` to generate a **Table 1** with minimal code

> **Note**
>
> These skills close the gap between raw R output and results you can actually put in a report.

The Problem We're Solving

This is what raw R output looks like:

```r
nhanes.samp %>%
  group_by(Gender) %>%
  summarise(
    n    = n(),
    mean = mean(BMI, na.rm = TRUE),
    sd   = sd(BMI, na.rm = TRUE)
  )
```

```
# A tibble: 2 × 4
  Gender     n  mean    sd
  <fct>  <int> <dbl> <dbl>
1 Female   102  26.6  9.11
2 Male      98  26.6  6.74
```

The Problem We're Solving

It's correct. But it is **not a result**:

- Column names are code names, not readable labels
- Numbers have too many decimal places
- There is no title or caption
- It would look terrible in a report

The principle for this module

A wall of raw R output is not a result. A table is. The gap between `print(df)` and a manuscript-ready table is smaller than you think.

Part 1: `knitr::kable()`

The Simplest Upgrade

`kable()` from the `knitr` package converts any data frame into a formatted HTML or PDF table in one line:

```r
nhanes.samp %>%
  group_by(Gender) %>%
  summarise(
    n    = n(),
    mean = round(mean(BMI, na.rm = TRUE), 1),
    sd   = round(sd(BMI, na.rm = TRUE), 1)
  ) %>%
  knitr::kable()
```

Gender	n	mean	sd
Female	102	26.6	9.1
Male	98	26.6	6.7

kable() with Options

```r
nhanes.samp %>%
  group_by(Gender) %>%
  summarise(
    n    = n(),
    mean = round(mean(BMI, na.rm = TRUE), 1),
    sd   = round(sd(BMI, na.rm = TRUE), 1)
  ) %>%
  knitr::kable(
    col.names = c("Sex", "N", "Mean BMI", "SD"),
    caption   = "BMI by sex in the NHANES sample",
    align     = c("l", "c", "c", "c")
  )
```

BMI by sex in the NHANES sample

Sex	N	Mean BMI	SD
Female	102	26.6	9.1
Male	98	26.6	6.7

Argument	Purpose
`col.names`	Override column names
`caption`	Table caption
`align`	Column alignment: `"l"`, `"c"`, `"r"`
`digits`	Decimal places (applies globally)

When to Use `kable()`

`kable()` is the right tool when you need a quick, clean table:

- Simple summary statistics
- Frequency tables from `tabyl()`
- Model coefficient tables from `tidy()`

For **publication-quality tables** with custom formatting, colors, or merged cells — use `gt`.

Part 2: The `gt` Package

Why `gt`?

`gt` ("grammar of tables") gives you fine-grained control over every element of a table — similar to how `ggplot2` gives you control over every element of a plot.

`kable()`

- One function
- Works for quick clean tables
- Limited formatting options

`gt`

- Layered grammar (like ggplot2)
- Column labels, formatting, spanners, footnotes
- Suitable for manuscripts and reports

Tip

`gt` tables render beautifully in HTML documents. For PDF output, additional setup is needed — HTML is the target format for this course.

A Minimal `gt` Table

Start with `gt()` and add layers:

```r
nhanes.samp %>%
  group_by(Gender) %>%
  summarise(
    n    = n(),
    mean = round(mean(BMI, na.rm = TRUE), 1),
    sd   = round(sd(BMI, na.rm = TRUE), 1)
  ) %>%
  gt()
```

Gender	n	mean	sd
Female	102	26.6	9.1
Male	98	26.6	6.7

Adding a Title and Column Labels

```r
nhanes.samp %>%
  group_by(Gender) %>%
  summarise(
    n    = n(),
    mean = round(mean(BMI, na.rm = TRUE), 1),
    sd   = round(sd(BMI, na.rm = TRUE), 1)
  ) %>%
  gt() %>%
  tab_header(
    title    = "BMI by Sex",
    subtitle = "NHANES sample (n = 200)"
  ) %>%
  cols_label(
    Gender = "Sex",
    n      = "N",
    mean   = "Mean BMI",
    sd     = "SD"
  )
```

RESULT: Adding a Title and Column Labels

BMI by Sex NHANES sample (n = 200)			
Sex	N	Mean BMI	SD
Female	102	26.6	9.1
Male	98	26.6	6.7

Formatting Numbers Inside the Table

`fmt_number()` formats specific columns — no more `round()` before the table. Now you keep raw values in the data and format only at the display stage.

```r
nhanes.samp %>%
  group_by(Gender) %>%
  summarise(
    n    = n(),
    mean = mean(BMI, na.rm = TRUE),
    sd   = sd(BMI, na.rm = TRUE)
  ) %>%
  gt() %>%
  tab_header(title = "BMI by Sex", subtitle = "NHANES sample (n = 200)") %>%
  cols_label(Gender = "Sex", n = "N", mean = "Mean BMI", sd = "SD") %>%
  fmt_number(columns = c(mean, sd), decimals = 1)
```

BMI by Sex

NHANES sample (n = 200)

Sex	N	Mean BMI	SD
Female	102	26.6	9.1
Male	98	26.6	6.7

Key `gt` Formatting Functions

Function	Purpose
`tab_header()`	Title and subtitle
`cols_label()`	Human-readable column names
`fmt_number()`	Format numeric columns (decimals, comma separators)
`fmt_percent()`	Format as percentages
`tab_footnote()`	Add footnotes to cells or columns
`tab_spanner()`	Group columns under a shared label
`cols_align()`	Column alignment
`tab_style()`	Bold, color, or style specific cells

A Multi-Variable Summary Table

```r
nhanes.samp %>%
  group_by(Gender) %>%
  summarise(
    n       = n(),
    bmi_m   = mean(BMI, na.rm = TRUE),
    bmi_sd  = sd(BMI, na.rm = TRUE),
    sbp_m   = mean(BPSysAve, na.rm = TRUE),
    sbp_sd  = sd(BPSysAve, na.rm = TRUE)
  ) %>%
  gt() %>%
  tab_header(
    title    = "BMI and Systolic Blood Pressure by Sex",
    subtitle = "NHANES sample (n = 200)"
  ) %>%
  cols_label(
    Gender = "Sex",    n = "N",
    bmi_m  = "Mean",   bmi_sd = "SD",
    sbp_m  = "Mean",   sbp_sd = "SD"
  ) %>%
  fmt_number(columns = c(bmi_m, bmi_sd, sbp_m, sbp_sd), decimals = 1) %>%
  tab_spanner(label = "BMI (kg/m²)",          columns = c(bmi_m, bmi_sd)) %>%
  tab_spanner(label = "Systolic BP (mmHg)",   columns = c(sbp_m, sbp_sd))
```

A Multi-Variable Summary Table

BMI and Systolic Blood Pressure by Sex

NHANES sample (n = 200)

		BMI (kg/m²)		Systolic BP (mmHg)	
Sex	N	Mean	SD	Mean	SD
Female	102	26.6	9.1	115.0	16.5
Male	98	26.6	6.7	119.3	17.5

Part 3: Table 1 with `gtsummary`

What Is a Table 1?

In clinical research, **Table 1** is the first table in a manuscript — it describes the sample by group:

Every clinical paper has one

"Table 1: Baseline characteristics of study participants by treatment group"

For continuous variables: mean (SD) or median (IQR)

For categorical variables: n (%)

Writing Table 1 by hand is tedious and error-prone. `gtsummary` generates it automatically.

`tbl_summary()` — Your First Table 1

```r
nhanes.samp %>%
  select(Gender, Age, BMI, BPSysAve) %>%
  tbl_summary()
```

Characteristic	N = 200[1]
Gender	
Female	102 (51%)
Male	98 (49%)
Age	31 (16, 50)
BMI	26 (21, 31)
Unknown	10
BPSysAve	114 (106, 127)
Unknown	30

[1] n (%); Median (Q1, Q3)

That's it. `tbl_summary()` detects variable types automatically:

- Continuous → median (IQR) by default
- Categorical → n (%)

Stratifying by a Group Variable

Add `by = Gender` to split the table into columns:

```
nhanes.samp %>%
  select(Gender, Age, BMI, BPSysAve) %>%
  tbl_summary(by = Gender)
```

Characteristic	Female N = 102[1]	Male N = 98[1]
Age	30 (16, 49)	31 (15, 50)
BMI	25 (20, 32)	26 (21, 31)
Unknown	8	2
BPSysAve	112 (103, 125)	118 (108, 128)
Unknown	19	11

[1] Median (Q1, Q3)

Customizing `tbl_summary()`

```r
nhanes.samp %>%
  select(Gender, Age, BMI, BPSysAve, HealthGen) %>%
  tbl_summary(
    by        = Gender,
    statistic = list(
      all_continuous()  ~ "{mean} ({sd})",
      all_categorical() ~ "{n} ({p}%)"
    ),
    digits    = all_continuous() ~ 1,
    label     = list(
      Age       ~ "Age (years)",
      BMI       ~ "BMI (kg/m²)",
      BPSysAve  ~ "Systolic BP (mmHg)",
      HealthGen ~ "Self-Reported Health"
    )
  ) %>%
  add_overall() %>%
  add_p() %>%
  bold_labels()
```

Customizing `tbl_summary()`

Characteristic	Overall N = 200[1]	Female N = 102[1]	Male N = 98[1]	p-value[2]
Age (years)	33.3 (21.1)	32.3 (21.2)	34.4 (21.0)	0.5
BMI (kg/m²)	26.6 (8.0)	26.6 (9.1)	26.6 (6.7)	0.5
Unknown	10	8	2	
Systolic BP (mmHg)	117.2 (17.1)	115.0 (16.5)	119.3 (17.5)	0.061
Unknown	30	19	11	
Self-Reported Health				>0.9
Excellent	21 (14%)	10 (14%)	11 (14%)	
Very Good	41 (27%)	20 (27%)	21 (27%)	
Good	57 (37%)	28 (38%)	29 (37%)	
Fair	33 (22%)	15 (20%)	18 (23%)	
Poor	1 (0.7%)	1 (1.4%)	0 (0%)	
Unknown	47	28	19	

[1] Mean (SD); n (%)

[2] Wilcoxon rank sum test; Fisher's exact test

`tbl_summary()` Key Arguments

Argument	Purpose
`by`	Stratify columns by a group variable
`statistic`	Control the summary format per variable type
`digits`	Decimal places for continuous variables
`label`	Human-readable variable labels
`include`	Which variables to include

Post-processing functions:

Function	Purpose
`add_overall()`	Add an “Overall” column
`add_p()`	Add p-values comparing groups
`bold_labels()`	Bold the row variable labels

Putting It All Together

Choosing the Right Tool

Situation	Tool
Quick check of output during analysis	`print()` / console
Simple table in a homework or report	`knitr::kable()`
Formatted multi-variable table for a report	`gt`
Table 1 with group comparisons	`gtsummary::tbl_summary()`
Model output (coefficients, CIs)	`gtsummary::tbl_regression()`

Tip

You don't need to memorize every function — you need to know which tool fits the situation, and know enough to look up the rest.

Key Takeaways

`kable()`

- `knitr::kable()` — one function, instantly clean
- `col.names`, `caption`, `align`, `digits`

`gt`

- `gt()` → `tab_header()` → `cols_label()` → `fmt_number()`
- `tab_spanner()` for grouped column headers
- Keep raw numbers in data; format only at display stage

`gtsummary`

- `tbl_summary(by = ...)` generates Table 1 automatically
- `statistic` controls mean (SD) vs median (IQR)
- `label` gives clean variable names
- `add_overall()` and `add_p()` for the finishing touches

Tip

The rule:

Raw R output is for you. A table is for your reader. Every number that appears in a report should be in a table or inline R — never copy-pasted.

References

`knitr::kable()`

- knitr documentation: kable

`gt`

- gt package documentation
- gt cookbook — Thomas Mock

`gtsummary`

- gtsummary documentation
- tbl_summary() tutorial